

News Topic Classification Problem

Matteo Ientile
Politecnico di Torino
Student id: s357948
s357948@studenti.polito.it

Abstract—In this report, we present a possible approach to the *News Topic* classification problem. In particular, after a first phase where additional statistics and features are extracted, the documents are transformed into a feature vector representation. The Term-Frequency Inverse-Document Frequency (TF-IDF) is the weighting scheme chosen. The proposed approach represents a significant improvement with respect to the naive baseline proposed.

I. PROBLEM OVERVIEW

The proposed competition is a classification problem that involves the categorization of approximately 100,000 instances describing online news articles, collected from multiple international news sources. The goal is to correctly classify the article category for each sample. The dataset is composed of two parts:

- a *development* set containing 80,000 instances, for which the label is provided
- an *evaluation* set with 20,000 instances.

The classification model is trained on the development set. Then, the model will be used to predict the labels for the news contained in the evaluation set.

We focus on the development set in order to make some considerations. The problem exhibits significant class imbalance as Table I shows. In particular, label 0 (*International News*) is the most frequent, while label 6 (*Health*) appears in less than 4% of instances.

Label	Category	Percentage (%)
0	International News	29.43
1	Business	13.24
2	Technology	13.95
3	Entertainment	12.47
4	Sports	10.72
5	General News	16.32
6	Health	3.88

TABLE I
LABEL DISTRIBUTION AS PERCENTAGE OF TOTAL SAMPLES

Moreover, the imbalance is present across the features. First, the total number of unique *sources* (i.e. publisher of the article) is 1,359, however the 20 most frequent cover almost 80% of the samples. We can visualize them in Figure 1 for a better understanding. Second, the *page rank* feature associated with the article source shows a skewed distribution. More precisely, 92% of the samples are associated with a page rank value equal to 5. The third feature we analyze is *timestamp*: each instance is associated with date and time of publication,

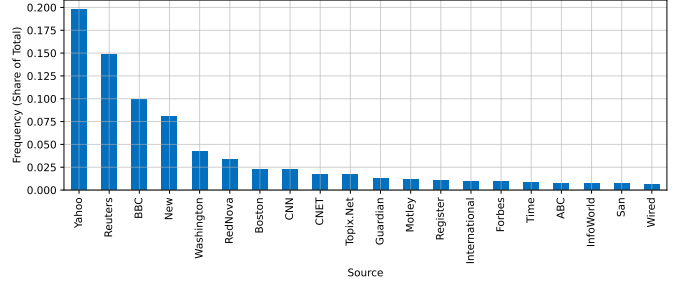


Fig. 1. 20 most frequent sources labels in the development set

however approximately 35% of samples of the development contain missing values (NaT).

Fourth and fifth features are *title* and *article*, containing respectively the title of the news and the full textual content of the article. We can observe the title length distributions in Figure 2.

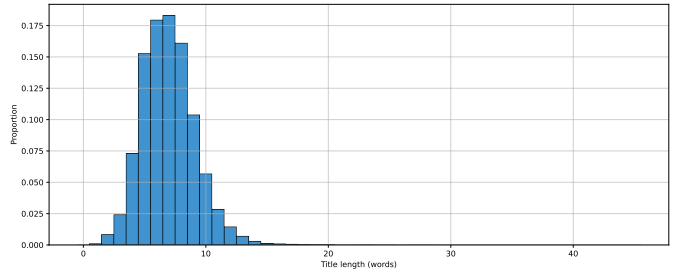


Fig. 2. Title: number of words distribution

We can visualize the number of words distribution for the article feature as well with Figure 3.

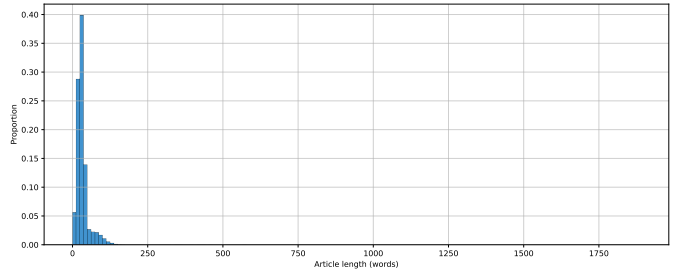


Fig. 3. Article: number of words distribution

Table II also summarizes the statistics for both the distributions: although the titles have a reduced standard deviation in term of length, the article lengths vary from 0 to 1,888 words.

Metric	Title Words	Article Words
Std. Dev.	2.26	39.44
Min	0	0
25%	5	22
50%	7	30
75%	8	38
Max	45	1888

TABLE II
DESCRIPTIVE STATISTICS FOR TITLE AND ARTICLE LENGTHS

The wide range of magnitude for the articles length must be taken into account in the next phases.

II. PROPOSED APPROACH

A. Preprocessing

Since almost 35% of timestamp data are missing values, we decide to drop the entire feature. The majority of instances belong to a narrow number of sources, as previously shown. Rather than encoding all the sources (and therefore adding 1,359 new dimensions), we decide to fix the "min_frequency" hyperparameter: the sources appearing at least a certain number of times are encoded, while the remaining ones are grouped together in one dimension as "infrequent". This choice produces two benefits:

- it lowers the dimensionality;
- the sources with low frequency are better captured by the model.

Title and article features both contain raw text, therefore we have to preprocess these features in order to train the model. We apply

- *tokenization*, i.e. the task of splitting the string into tokens and also removing the punctuation;
- *stop word elimination*, that consists of deleting from the list of tokens the most common words across all the documents.

Contrary to typical text preprocessing pipelines, we apply neither case normalization nor lemmatization. Two main reasons support this decision

- Case normalization consists of converting all tokens to lower or upper case. In this context, combination of stop word elimination and case normalization deletes some fundamental key words for topic recognition. For example, "US" (United States) and "us" become the same word according to case normalization, and then deleted since "us" is a stop word.
- Lemmatization consists of performing a full morphological analysis in order to only keep the words belonging to the official dictionary (e.g. "is" becomes "be"). However, especially for documents written in English, the lemmatization provides at most modest benefits [1]. This happens because particular inflection forms rely on specific contexts, and therefore have different importance.

Furthermore, we adopted a personalized list of stop words: in addition to the one provided by default in *scikit-learn*, we also included the same words but with capital letter. This choice aims to manage edge cases: for instance, the word "US" will be kept, while "Us" and "us" will be eliminated.

To obtain a vector features representation, we use the *weighted document representation* approach. In particular, we adopt the *Term Frequency - Inverse Document Frequency* (TF-IDF) weighting scheme. As a result, each word (token) of title and article features is converted to a number that reflects how important the word is to a document [2].

B. Model selection

The following algorithms have been tested:

- *Complement Naive Bayes*: in the context of text classification, naive Bayes classifiers typically perform well [1]. A naive Bayes classifier estimates the class-conditional probability by assuming that the attributes are conditionally independent, given the class label [3]. The classical approach uses the Multinomial Bayes Classifier (MNB), however in case of imbalanced training data and documents with different lengths it struggles. Since the development set has both these problems, we decided instead to test the *Complement Naive Bayes* (CNB) [4], able to overcome the problems of MNB. Weights for the CNB algorithm are computed using data from all classes except one, while MNB works in the opposite way. This corrects the bias towards majority classes, a problem that affects the Multinomial Naive Bayes.
- *SVM*: this model has shown several promising results in the context of text categorization. Moreover, this algorithm performs well with high-dimensional data and avoids the curse of dimensionality problem [3]. Furthermore, SVM is able to manage class imbalance. This model transforms the data and identifies the hyperplane (decision boundary) that maximizes the margin between classes [3]. This model requires feature normalization.

To decide which model to use, we first performed a random search to explore the domain of possible hyperparameters. Then, we selected the exact hyperparameters to test with a grid search.

C. Hyperparameters tuning

To ensure a fair comparison, we decoupled the tuning of the two models, as they require different feature representations. The procedure is the following:

- 1) Perform a random search for the SVM pipeline (tuning vectorizer and model parameters).
- 2) Perform a random search for the Complement Naive Bayes pipeline (tuning vectorizer and model parameters).
- 3) Fix the *ColumnTransformer* hyperparameters (obtained from the random search), and perform a grid search to tune more precisely the models hyperparameters.

We used a 80/20 (Train/Test) split. We also built separate vectorizers for the title and article features, based on the assumption that the density of informative keywords is typically higher in titles than in article bodies [1].

III. RESULTS

The best configuration selected for the SVM model (text vectorizer and model hyperparameters) is summarized in Table III. Hyperparameters left at their default values [5] are omitted for brevity. Using two different text vectorizers

	Hyperparameter	Value
SVM Model	<i>C</i>	0.1
	<i>loss</i>	squared_hinge
	<i>class_weight</i>	balanced
Title Vectorizer	<i>ngram_range</i>	(1, 3)
	<i>max_df</i>	0.5
	<i>min_df</i>	2
	<i>lowercase</i>	False
	<i>strip_accents</i>	ascii
	<i>stop_words</i>	customized*
Article Vectorizer	<i>ngram_range</i>	(1, 2)
	<i>max_df</i>	0.6
	<i>min_df</i>	15
	<i>lowercase</i>	False
	<i>strip_accents</i>	ascii
	<i>stop_words</i>	customized
	<i>sublinear_tf</i>	True
OneHotEncoder	<i>min_frequency</i>	50
	<i>handle_unknown</i>	infrequent_if_exist

TABLE III
BEST CONFIGURATION - SVM. (*NOTE: "CUSTOMIZED" STOP WORDS REFER TO THE LIST DESCRIBED IN THE PREPROCESSING SECTION.)

provides the anticipated benefit: the title vectorizer adopts a wider *ngram_range* attempting to capture "local context" and keywords [1]. On the other hand, the article vectorizer acts as a noise-filtering mechanism with a narrower *ngram_range* and higher *max_df* threshold.

The best configuration selected for the Complement Naive Bayes model (text vectorizer and model hyperparameters) is summarized in Table III. The Complement Naive Bayes model

	Hyperparameter	Value
CNB Model	<i>classifier_alpha</i>	0.15
	<i>classifier_norm</i>	True
Title Vectorizer	<i>ngram_range</i>	(1, 3)
	<i>min_df</i>	1
	<i>lowercase</i>	False
	<i>stop_words</i>	customized
	<i>use_idf</i>	True
	<i>norm</i>	l2
	<i>sublinear_tf</i>	False
Article Vectorizer	<i>ngram_range</i>	(1, 3)
	<i>min_df</i>	3
	<i>lowercase</i>	False
	<i>stop_words</i>	customized
	<i>use_idf</i>	False
	<i>norm</i>	l2
	<i>sublinear_tf</i>	True
OneHotEncoder	<i>min_frequency</i>	50
	<i>handle_unknown</i>	infrequent_if_exist

TABLE IV
BEST CONFIGURATION - COMPLEMENT NAIVE BAYES

selected *use_idf=True* for the Article feature, relying on raw term counts. This confirms the underlying theory [4] that applying IDF and length normalization to the Complement Naive Bayes algorithm significantly improves stability on documents with varying lengths.

The SVM configuration proved to be superior. The results are shown in Table III.

	Test Set F1-Macro	Public LB F1-Macro
Linear SVM	0.717	0.731
Complement Naive Bayes	0.686	0.692

TABLE V
PERFORMANCE COMPARISON ON TEST SET & PUBLIC LEADERBOARD

Since these are the first scores obtained using the pipeline described, there is likely no overfitting. Moreover, the low *C* value also contributes to mitigating the risk of overfitting and improves the model generalization on unseen data [6].

In order to have a more detailed view of the results, we provide the confusion matrices obtained with SVM and CNB on the test set. As observed in Figure 4, both models

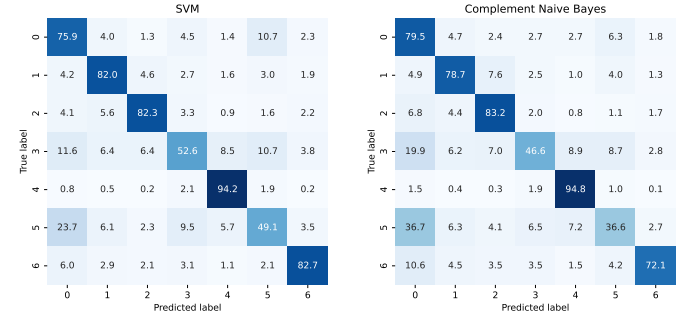


Fig. 4. Confusion Matrices on Test Set: SVM (left) vs. CNB (right)

demonstrate robustness against the severe class imbalance, particularly for Class 6 (*Health*), which is correctly identified despite representing less than 4% of the development data.

However, a consistent error pattern emerges where Class 3 (*Entertainment*) and Class 5 (*General News*) are frequently misclassified as Class 0 (*International News*). This suggests a semantic overlap where specific topics (Class 3/5) are linguistically similar to the "International News" category. We can reasonably expect a similar behaviour for the evaluation set.

IV. DISCUSSION

The proposed approach obtains results that significantly improve upon the naive baseline score present in the public leaderboard (f_1 score = 0.443). Although SVM has shown better scores overall, the CNB model still obtains acceptable results considering the reduced computational effort required for hyperparameter tuning.

The following approaches can be considered in order to improve the results further:

- Extending the customized stop word list could be considered to address the misclassification problems across

labels 0, 3 and 5. In particular, in the context of News Topic classification, several "clickbait" terms (especially in titles) are shared across the articles [7]. Therefore, their removal could improve the model's ability to focus on keywords that discriminate specific categories.

- Exploring Deep Learning approaches may lead to significant improvements [8]. In particular, it has been shown that Convolutional Neural Networks (CNN), BERT [9] and other transformer-based models are the current state of the art in text classification.

REFERENCES

- [1] C. D. Manning, *Introduction to information retrieval*. Syngress Publishing., 2008.
- [2] P. Bafna, D. Pramod, and A. Vaidya, "Document clustering: Tf-idf approach," in *2016 International conference on electrical, electronics, and optimization techniques (ICEEOT)*, pp. 61–66, IEEE, 2016.
- [3] P.-N. Tan, M. Steinbach, and V. Kumar, *Introduction to data mining*. Pearson Education India, 2016.
- [4] J. D. Rennie, L. Shih, J. Teevan, and D. R. Karger, "Tackling the poor assumptions of naive bayes text classifiers," in *Proceedings of the 20th international conference on machine learning (ICML-03)*, pp. 616–623, 2003.
- [5] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. VanderPlas, A. Joly, B. Holt, and G. Varoquaux, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pp. 108–122, 2013.
- [6] P. Lameski, E. Zdravevski, R. Mingov, and A. Kulakov, "Svm parameter tuning with grid search and its impact on reduction of model over-fitting," in *Rough Sets, Fuzzy Sets, Data Mining, and Granular Computing: 15th International Conference, RSFDGrC 2015, Tianjin, China, November 20-23, 2015, Proceedings*, pp. 464–474, Springer, 2015.
- [7] A. Chakraborty, B. Paranjape, S. Kakarla, and N. Ganguly, "Stop clickbait: Detecting and preventing clickbaits in online news media," in *2016 IEEE/ACM international conference on advances in social networks analysis and mining (ASONAM)*, pp. 9–16, IEEE, 2016.
- [8] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning-based text classification: a comprehensive review," *ACM computing surveys (CSUR)*, vol. 54, no. 3, pp. 1–40, 2021.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.